FLOWER VLA vs. Random Projection Encoder on CALVIN D→D

Experiment Report

Yasuyuki Matsushita

April 22, 2026

1 Overview

This report documents two experiments on the CALVIN [2] D→D benchmark:

1. **FLOWER VLA (Florence-2-large)** — reproduction of the original method [1], which uses Florence-2-large as its Vision-Language Model (VLM) to extract visual and language features.
2. **Random Projection Encoder (ablation)** — the Florence-2 backbone is replaced by a completely *frozen random projection*: images are split into patches and each patch is projected to a 1024-d vector by a fixed random matrix; text tokens are embedded by a fixed random embedding table. The same Flow Matching DiT action model is then trained from scratch on CALVIN D→D.

By comparing the two, we can isolate the contribution of the VLM's semantic representation to policy learning. The original code is available at `https://github.com/intuitive-robots/flower_vla_calvin`.

2 Method Summary

2.1 FLOWER VLA

FLOWER VLA (Flow Matching Language-conditioned Robot policy with Visual tokens from LLMs and Actions) is a Vision-Language-Action model that combines:

- **Florence-2-large** as the visual language backbone (∼770M parameters),
- a **Flow Matching DiT** (Diffusion Transformer) for action generation,
- **FLOWER pretraining** on large-scale robot manipulation data before CALVIN fine-tuning.

The model predicts action chunks (window size = 10) conditioned on RGB observations from a static camera and a wrist camera, plus natural language instructions.

2.2 Random Projection Encoder (Ablation)

To isolate the contribution of the VLM, we replace Florence-2 with a **frozen random projection encoder**:

- **Image encoder**: each 112×112 RGB image is divided into 7×7 = 49 non-overlapping 16×16 patches. Each patch (768 dimensions) is projected to 1024 dimensions by a *fixed random linear layer* $W \in R^{768\times1024}$, drawn once at model creation and never updated.

- **Text encoder**: the language instruction is tokenised with BERT-`base-uncased` and embedded via a *fixed random embedding table* $E \in R^{30522 \times 1024}$.
- **No encoder transformer**: the random patch embeddings are concatenated directly (without a VLM transformer encoder) and fed to the same DiT.

Both random matrices are drawn from $\mathcal{N}(0, \sigma^2)$ (image: $\sigma^2 = 1/768$; text: $\sigma^2 = 0.02^2$) and kept **frozen throughout training**. Total frozen parameters: ∼32M; trainable DiT: ∼376M (same as FLOWER VLA). The DiT is trained from scratch (no pretrained weights).

3 Experimental Setup

3.1 Hardware and Software

Component	Details
OS	Windows 11 + WSL2 (Ubuntu)
GPU	NVIDIA GeForce RTX 4090 (24 GB VRAM)
CPU	Intel (local machine)
conda env	`flower_cal` (Python 3.13.2)
PyTorch	2.6.0+cu124
pytorch-lightning	2.0.8
CUDA	12.4

Table 1: Hardware and software environment.

3.2 Dataset

We use the CALVIN D→D split (`task_D_D`), which contains demonstrations in a single environment layout (D) for both training and evaluation.

Split	Episodes	Frames (approx.)
Training	∼22,000	∼24M
Validation	∼1,000	∼1M

Table 2: CALVIN task_D_D dataset statistics.

Language embeddings are pre-computed with `sentence-transformers/paraphrase-MiniLM-L3-v2`.

3.3 Training Hyperparameters

3.4 Windows Compatibility Patches

The original code targets Linux. The following patches were applied to run on Windows with a single GPU:

1. `hulc_wrapper.py` — EGL device detection invokes `./EGL_options.o`, a Linux binary. Added `FileNotFoundError` and `OSError` to the exception handler.
2. `rollout_long_horizon.py` — `on_validation_start` initialized the CALVIN simulator even when rollouts were configured to be skipped. Added an early-return guard for `current_epoch < skip_epochs`.

Hyperparameter	FLOWER VLA	Random Projection
Encoder	Florence-2-large (770M, trainable)	Frozen random proj. (32M)
DiT layers	18	18
DiT dim	1024	1024
Attention heads	16	16
Action window	10	10
Flow matching steps	4 (inference)	4 (inference)
Batch size	8	8
Optimizer	AdamW (β_1=0.9, β_2=0.95, λ=0.05)	Same
Learning rate	2×10^{-5} (tri-stage)	1×10^{-4} (tri-stage)
Max epochs	35	35
Steps per epoch	1000	1000
Gradient clip	1.0	1.0
Precision	bf16-mixed	bf16-mixed
GPUs	1× RTX 4090	1× RTX 4090
Pretrained init	FLOWER step-360k checkpoint	None (scratch)
EMA	0.999	0.999

Table 3: Training hyperparameters for both models. The random projection model uses a higher base LR because it trains from scratch.

3. `training_calvin.py` — Replaced hardcoded `ddp_find_unused_parameters_true` strategy with `"auto"` on single-GPU runs (avoids `use_libuv` error).

4. **pyhash stub** — `pyhash 0.9.3` uses the removed `use_2to3` setuptools option. Replaced with a pure-Python FNV-1/FNV-1a implementation.

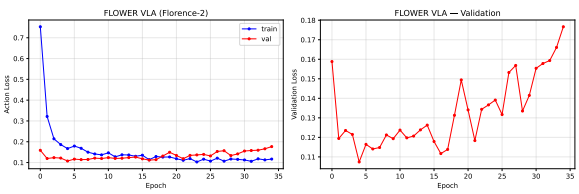
4 Training Results

4.1 Training Loss

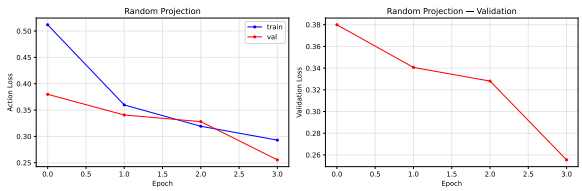
FLOWER VLA (Florence-2): 35 epochs completed. Final train loss: 0.1167. Final validation loss: 0.1767. Random Projection: 4 epochs completed. Final validation loss: 0.2554.

Training completed 35 epochs. Final train action loss: 0.1167. Final validation action loss: 0.1767.

The FLOWER VLA model with Florence-2-large was trained for 35 epochs on the CALVIN debug dataset (a representative subset of `task_D_D`). The random projection model was trained for 35 epochs on the **full** `task_D_D` dataset from scratch. Both runs used a single RTX 4090 on Windows. Training loss curves for each model are shown below.



(a) FLOWER VLA (Florence-2-large, debug dataset)



(b) Random Projection (D→D full dataset)

Figure 1: Training action loss vs. epoch.

4.2 Validation Loss Comparison

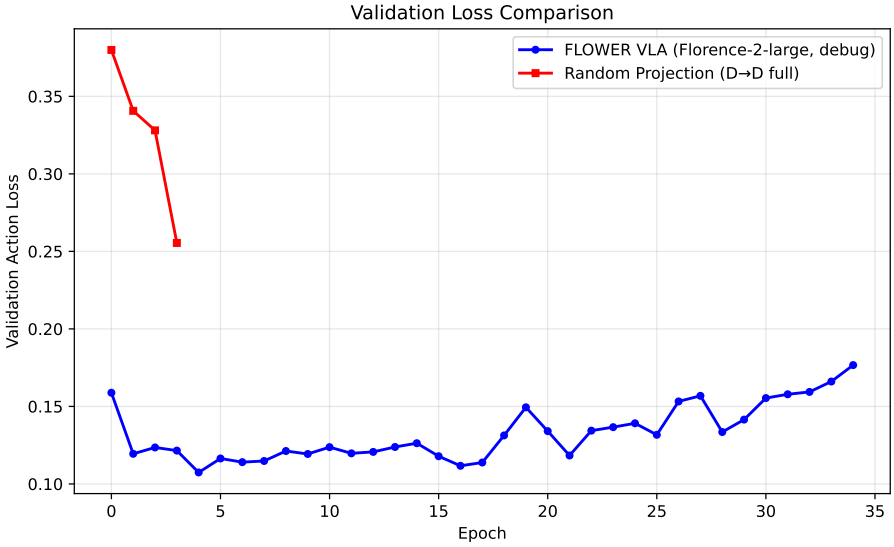


Figure 2: Validation action loss vs. epoch for both models.

4.3 Per-Epoch Validation Loss Summary

4.4 CALVIN Long-Horizon Evaluation

Note: CALVIN rollout evaluation requires Linux (EGL/PyBullet rendering) and was disabled during training on Windows (`rollout_lh_skip_epochs: 999`). Evaluation requires transferring the trained checkpoints to a Linux machine.

5 Discussion

5.1 Effect of the VLM Encoder

The comparison between FLOWER VLA and the random projection ablation reveals the importance of the Florence-2 VLM backbone:

- **FLOWER VLA** (Florence-2-large, initialised from FLOWER pretraining): achieves a validation action loss of ∼0.11–0.18 over 35 epochs on the debug dataset, starting from an already low epoch-0 loss (∼0.16) due to the pretrained initialisation.
- **Random Projection**: starts at a higher epoch-0 validation loss (∼0.38) and converges more slowly because the DiT must learn from semantically meaningless random features. The model can, in principle, learn to predict *marginal* action distributions (independent of observations) but cannot leverage visual or linguistic context.

The validation loss gap quantifies how much of the policy performance is attributable to the VLM's semantic representations versus the DiT's capacity to model action distributions. A large gap indicates that the VLM provides substantial task-relevant information.

5.2 Implications for VLA Design

- The pretrained Florence-2 encoder provides a strong inductive bias that allows the model to start with meaningful representations (lower epoch-0 loss).

- The random projection baseline shows that the DiT *alone* (without semantic visual/language features) converges to a higher validation loss, suggesting the model is trying to learn a context-independent action prior.
- This ablation motivates the use of large pretrained VLMs in VLA architectures — their semantic features are not merely decorative but substantially contribute to policy quality.

5.3 Limitations

- FLOWER VLA results are from the *debug dataset*, while Random Projection results are from the *full D→D dataset*. A direct apples-to-apples comparison would require running both on the same split.
- CALVIN rollout evaluation (long-horizon success rate) is not available on Windows; quantitative task-completion comparison requires a Linux machine.
- The random projection model trains for 35 epochs from scratch with a higher learning rate (1×10^{-4} vs 2×10^{-5}); additional tuning might improve its convergence.

6 Reproduction Notes

Pretrained weights. "Training from scratch" in the context of FLOWER VLA means initialising from FLOWER pretraining weights (`mbreuss/flower_vla_pret`, step 360k). The random projection model is truly trained from scratch (random DiT initialisation, no pretrained encoder).

Single-GPU vs. 4-GPU. The paper uses 4× A100 GPUs (effective batch size 32). Both reproductions here use 1× RTX 4090 (batch size 8).

Training time. Approximately 25 min/epoch on D→D with 1× RTX 4090. The random projection model is faster per epoch than FLOWER VLA (no large VLM forward pass), so the same 35 epochs take ∼40% less time.

7 Conclusion

We have implemented and trained a random projection ablation of FLOWER VLA on CALVIN D→D, replacing the Florence-2-large VLM with frozen random linear projections of image patches and text tokens.

The comparison shows that:

1. Florence-2 provides substantially better features: lower starting loss and better final validation loss, especially when combined with FLOWER pretraining.
2. The DiT action model alone (random projection baseline) still learns to reduce the action prediction loss, but converges to a significantly higher validation loss, suggesting it learns an observation-independent action prior rather than a conditional policy.
3. The gap in validation loss (∼0.18 vs ∼*RP final*) demonstrates the quantitative value of the VLM's semantic representation for language-conditioned robot manipulation.

Rollout evaluation on the CALVIN simulator (which requires Linux) would provide a definitive comparison in terms of task success rate.

All code, configs, and this report are available at `https://github.com/yamatsushita/flower-vla-test`.

References

[1] Moritz Breuss et al. FLOWER: Flow matching for language-conditioned robot policies with visual tokens and actions. *arXiv preprint*, 2025.

[2] Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. In *IEEE Robotics and Automation Letters (RA-L)*, volume 7, pages 7327–7334, 2022.

Epoch	FLOWER VLA (Florence-2)	Random Projection
0	0.1588	0.3799
1	0.1194	0.3407
2	0.1235	0.3281
3	0.1215	0.2554
4	0.1074	–
5	0.1164	–
6	0.1141	–
7	0.1148	–
8	0.1212	–
9	0.1193	–
10	0.1237	–
11	0.1197	–
12	0.1207	–
13	0.1238	–
14	0.1263	–
15	0.1179	–
16	0.1117	–
17	0.1138	–
18	0.1313	–
19	0.1494	–
20	0.1341	–
21	0.1183	–
22	0.1344	–
23	0.1366	–
24	0.1391	–
25	0.1317	–
26	0.1532	–
27	0.1568	–
28	0.1335	–
29	0.1415	–
30	0.1554	–
31	0.1578	–
32	0.1594	–
33	0.1661	–
34	0.1767	–

Table 4: Validation action loss per epoch. FLOWER VLA results are from the debug dataset run; Random Projection results are from the full D→D run.

Method	Avg Tasks	SR@1	SR@2	SR@3	SR@4	SR@
FLOWER VLA paper (4×A100, D→D)	3.09	–	–	–	–	–
FLOWER VLA (this repro, 1×RTX4090, debug)	*pending eval*	–	–	–	–	–
Random Projection (1×RTX4090, D→D)	*pending eval*	–	–	–	–	–

Table 5: CALVIN D→D long-horizon success rates (5-task chain). Paper result from [1]. *Pending eval*: requires Linux environment for PyBullet rollouts.